PLATFORM SPECIFICATION · INTERNAL

E.C.H.O

Erez Compute Host Operations

ABSTRACT

E.C.H.O is a web-based GPU orchestration platform built on Kubernetes. It provides a unified interface for submitting, scheduling, monitoring and managing AI compute operations across three workload types: long-running inference services, time-bound research environments, and time-bound training jobs.

PostgreSQL is the central source of truth for requests, queues, policies, metadata and audit history. E.C.H.O workers reconcile that desired state into Kubernetes, while Kueue manages GPU quota admission and the Kubernetes scheduler places workloads onto GPU nodes. Organizational users and groups enforce access, priorities, limits and fair resource sharing.

This document specifies the architecture, data model, protocols, lifecycle and operational behaviour of the platform. It is normative for implementations.

DOCUMENT	ECHO-SPEC-001
VERSION	1.0
STATUS	Draft Standard — normative for implementation
DATE	24 August 2026
SUPERSEDES	All prior E.C.H.O design material

Table of Contents

1 Introduction

1.1 Purpose and Scope

E.C.H.O simplifies access to shared AI infrastructure, maximizes GPU utilization, prevents conflicting allocations, and provides reliable lifecycle management from submission and admission through execution, monitoring, cancellation, expiration and completion.

This specification covers the control plane, the worker pool, the persistence model, the protocols between them, and the behaviour of each workload type. It does not specify the internal implementation of user-supplied workloads, the training frameworks they use, or the physical provisioning of GPU hardware.

1.2 Workload Models

E.C.H.O supports two workload models, and the distinction runs through the whole specification.

Bring-your-own image — Research and Training
: The user supplies a complete container image containing the required tools and code. E.C.H.O treats the image as opaque: it validates the *request*, allocates the requested compute, and runs it as a time-bound session or job. E.C.H.O does not inspect or construct the image.

Managed service — Inference
: E.C.H.O owns and maintains a versioned, prebuilt inference runtime image containing vLLM and the required platform components. The client supplies a declarative inference specification and receives a stable inference endpoint. The client never receives access to the underlying Pod.

1.3 Requirements Language

The key words **MUST**, **MUST NOT**, **REQUIRED**, **SHALL**, **SHALL NOT**, **SHOULD**, **SHOULD NOT**, **RECOMMENDED**, **MAY** and **OPTIONAL** in this document are to be interpreted as described in RFC 2119.

These keywords carry their normative meaning only in Sections 3 through 15, which define conformance surface. Sections 1 and 2 and the appendices are descriptive. Numbered requirement blocks (`R-nn`) collect the obligations that an implementation is held against; Appendix B lists them in one place.

1.4 Terminology

Operation
: A single logical unit of user intent — one training job, one research environment, or one inference model. Identified by an operation ID and durable for its whole life.

Attempt
: One physical execution of an operation. An operation may have several attempts over its life; each has a deterministic identity of the form `echo-<operation-id>-a<attempt-number>`.

Revision

A version of an operation's *desired state*, created when the user or policy changes intent. Not a record of execution.

Placement

The cluster, and for pinned inference the node, on which an operation runs. Observed from Kubernetes, never chosen by E.C.H.O.

Plan

The immutable execution plan produced by the planner: target cluster, namespace, queue, priority, GPU flavor and attempt identity.

Project

The unit of tenancy. Owns namespaces, quotas, storage and access. Each project has one namespace per workload type in each cluster.

Fencing token

The pair `(lease_owner, claim_epoch)` that a worker must present on every completion write, so that a stalled worker's late write is rejected.

Control plane

The API, planner, policy engine and expiry/recovery scanner. Reads and writes PostgreSQL; never calls the Kubernetes API.

Worker

A cluster-agnostic reconciler that claims work from PostgreSQL and materializes it in a target Kubernetes cluster.

2 Architecture

2.1 System Ownership

Every concern in the platform has exactly one authority. This table is the reference for the whole document; where a later section appears to give a concern to a different component, this table wins.

CONCERN	AUTHORITY
User request and desired state	PostgreSQL writer
Workflow / reconciliation queue	PostgreSQL writer
User, group and policy snapshot	PostgreSQL
Target cluster and resource plan	PostgreSQL
Logical GPU admission	Kueue
Pod-to-node placement	Kubernetes scheduler
Actual workload state	Kubernetes API
Reporting and dashboards	PostgreSQL read replicas
GPU telemetry	Prometheus / DCGM, summarized into PostgreSQL

> **AUTHORITY, PRECISELY**
>
> PostgreSQL is the source of truth for *intent*: what the user asked for, what policy allowed, what the plan committed to, and what the queue owes. It also *reports* runtime state, but that report is a projection of Kubernetes, not an independent truth. Once an `EchoExecution` exists, Kubernetes is authoritative for what is actually running.

2.2 Components

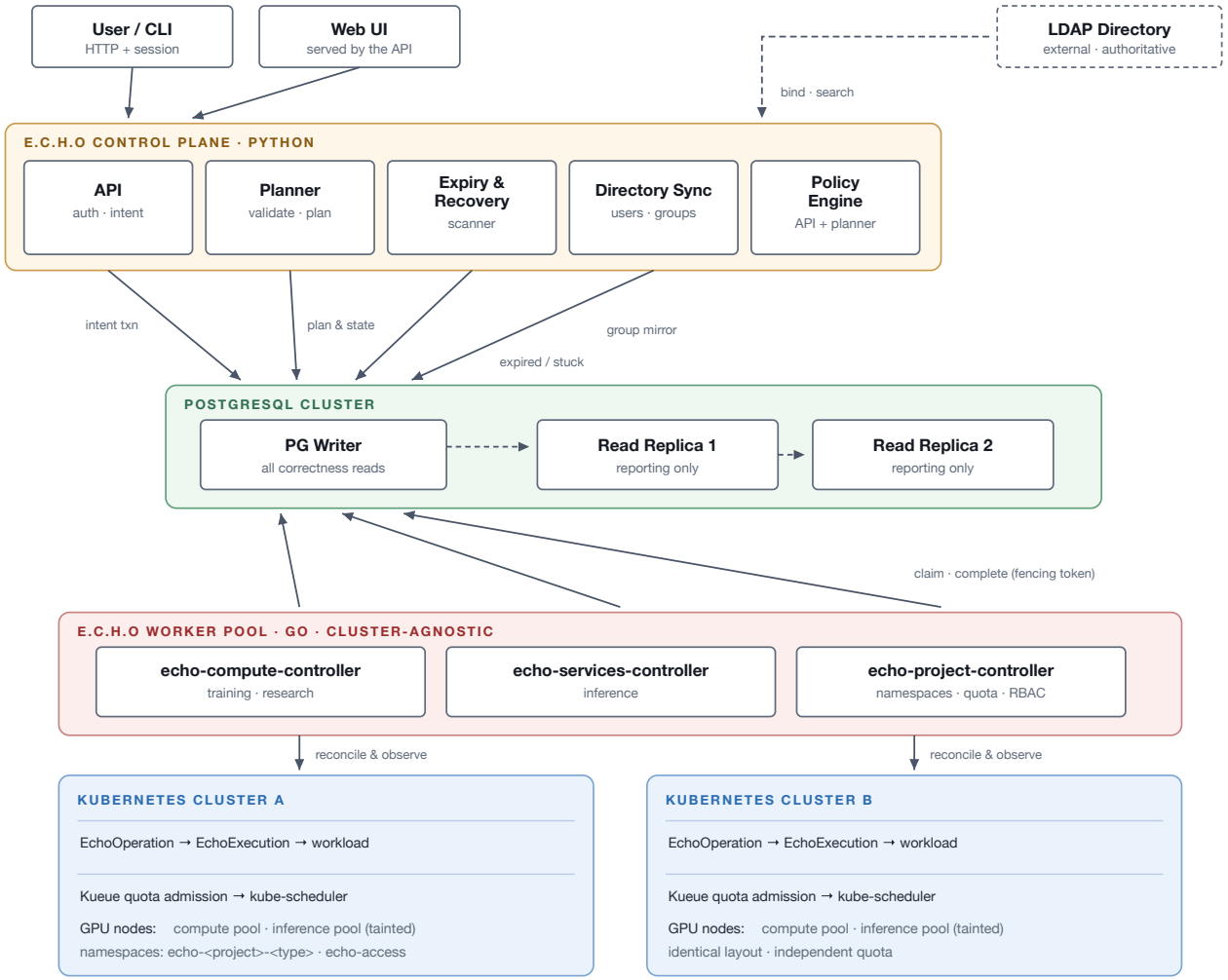

Figure 1. Component architecture. Every control-plane component writes to the PostgreSQL writer and none of them calls Kubernetes. Workers are the sole bridge: they claim work from PostgreSQL and reconcile it into a target cluster, then project observed state back.

2.2.1 Control plane

One deployable package with three entrypoints, each scaled independently:

`api`
: Authenticates requests, evaluates policy, and writes user intent in a single transaction. Serves the web UI's static assets and its read queries.

`planner`
: Claims unplanned operations, validates the requested placement against policy, and stores the immutable execution plan.

`scanner`
: Finds expired and stuck operations and drives them to their terminal or recovery state.

All three claim work with the same row-locking protocol as the worker (Section 8.1), so all three scale horizontally and none requires leader election.

2.2.2 Worker pool

Workers are cluster-agnostic: any worker can materialize any plan in that plan's target cluster. They are deployed one per controller identity, not one per cluster, and each process therefore holds a controller-runtime manager and cache per registered cluster.

> **R-01**
> A worker **MUST NOT** hold a PostgreSQL transaction open while calling the Kubernetes API.

> **R-02**
> A worker process holding managers for multiple clusters **MUST** isolate them, so that one cluster's reconnect storm or cache failure cannot starve reconciliation for the others. Bounded per-cluster work queues and per-cluster circuit breaking are **RECOMMENDED**.

2.2.3 Persistence

A single PostgreSQL writer, with read replicas for reporting.

> **R-03**
> Read replicas **MUST NOT** be used for correctness decisions. They serve reporting and dashboards only. Any decision that admits, transitions, claims or completes an operation **MUST** read from the writer.

2.3 Implementation Stack

LAYER	TECHNOLOGY
Control plane	Python + FastAPI; SQLAlchemy as the query layer; Alembic runs hand-written migrations
Worker pool	Go + controller-runtime
Web UI	React + TypeScript + Vite, built to static assets and served by the API
Database	PostgreSQL; lifecycle enforced in-database by a transition function and a trigger
Admission	Kueue
Custom resources	`EchoOperation` (operation-scoped) parents `EchoExecution` (attempt-scoped)
Directory	LDAP; the API binds and issues its own revocable server-side session

The language boundary sits exactly on the database. The control plane never calls Kubernetes, so it has no need of a Kubernetes client; the worker is the only component that does, and it lives where the mature controller tooling is. Because the two sides communicate only through PostgreSQL, the schema — not an RPC contract — is the interface between them.

3 Cluster Topology and Isolation

3.1 Multi-Cluster Model

E.C.H.O is a multi-cluster control plane. Every registered cluster carries the same namespace layout, the same controller identities and the same Kueue structure, so a plan is portable in shape even though its target is fixed. Kueue quota is per-cluster; there is no cross-cluster borrowing.

> **R-04**
>
> Each registered cluster **MUST** have an entry in the cluster registry recording its identifier, API endpoint, credential reference and available GPU flavors. An operation **MUST NOT** be planned onto a cluster absent from the registry.

3.2 Namespace Architecture

Tenancy is per project. Each project receives one namespace per workload type in each cluster, named `echo-<project>-<type>`. A single additional namespace, `echo-access`, holds the controller identities and nothing else.

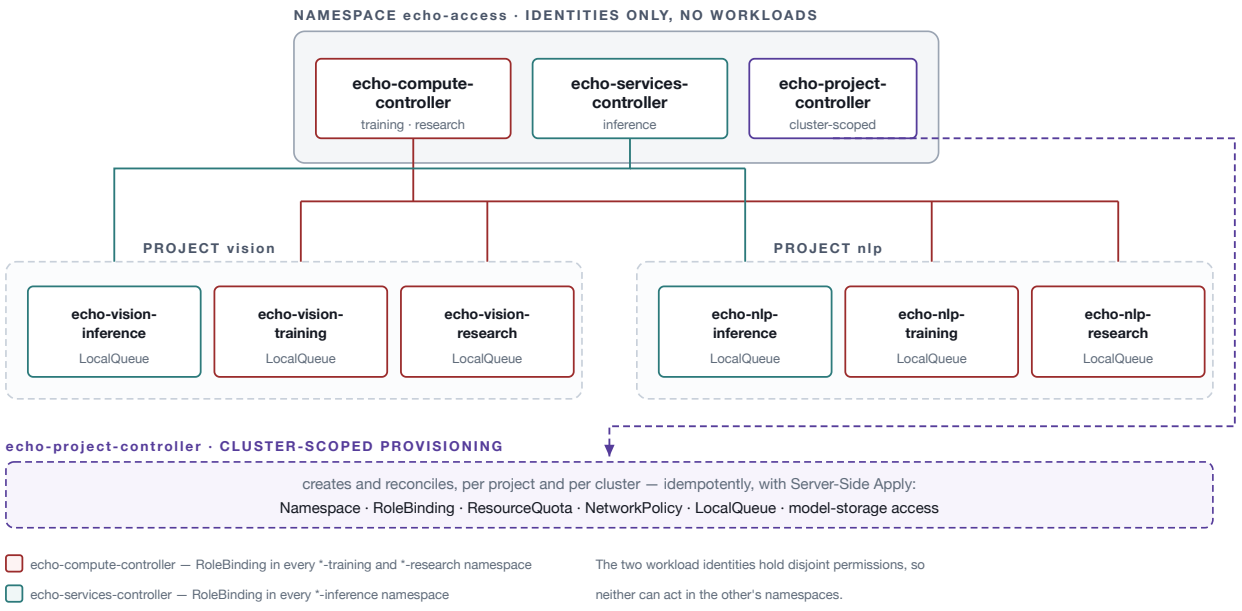

Figure 2. Namespace and identity model within one cluster. The controller identities live apart from the namespaces they act on, and each is bound only to the namespaces its workload type occupies — so a fault in the compute reconciler cannot reach an inference service.

> **R-05**
>
> Workload namespaces **MUST** be named `echo-<project>-<type>` where *type* is one of `training`, `research` or `inference`.

R-06

Selection of E.C.H.O-managed objects **MUST** use labels, not namespace-name parsing. Every managed object **MUST** carry:

```yaml
metadata:
  labels:
    echo.erez.io/project: vision
    echo.erez.io/resource-type: research
    echo.erez.io/managed: "true"
```

3.2.1 What the namespace boundary carries

Per-project namespaces are the unit that carries GPU, CPU and memory quota; Secrets and PVCs; NetworkPolicy; Kubernetes RBAC; the Kueue `LocalQueue`; workload visibility and logs; and deletion blast radius. A shared per-type namespace would forfeit all of these.

> **NAMESPACES ARE AN AUTHORIZATION BOUNDARY, NOT A RESOURCE ONE**
>
> A namespace does not stop a training Pod from landing on the same node as an inference Pod and starving it on CPU, memory or PCIe bandwidth, and it gives no control-plane isolation whatsoever. Node separation (Section 3.4) is what provides resource isolation; the namespace provides the authorization boundary.

3.3 Controller Identities

Three ServiceAccounts live in `echo-access`. Their permissions are disjoint by design.

IDENTITY	BOUND TO	PRINCIPAL VERBS
`echo-compute-controller`	every `*-training` and `*-research` namespace	Job, StatefulSet, Pod, PVC, ConfigMap
`echo-services-controller`	every `*-inference` namespace	Deployment, Service, HPA, ConfigMap, Ingress/HTTPRoute, model-storage Secret
`echo-project-controller`	cluster-scoped	Namespace, RoleBinding, ResourceQuota, NetworkPolicy, LocalQueue

R-07

The three controller identities **MUST** hold disjoint permissions. `echo-project-controller` **MUST NOT** hold verbs on Pods, Jobs or Deployments, and the two workload identities **MUST NOT** hold verbs on Namespaces or RoleBindings. An identity that can create namespaces must not be able to run workloads, and an identity that runs workloads must not be able to mint its own permissions.

> **R-08**
>
> Roles and RoleBindings **MUST** be created in the target namespace with subjects referring to the ServiceAccounts in `echo-access`.

Because a Pod carries exactly one ServiceAccount, each identity corresponds to a separate worker Deployment. This is desirable independently: inference reconciliation is low-volume and steady, while training is bursty, and the two should scale apart.

3.4 Node Pools

> **R-09**
>
> Inference nodes **MUST** be tainted, and only inference workloads **MUST** tolerate that taint. Inference workloads **MUST** run under a non-preemptible priority class.

Training and research share the remaining pool. Their schedules are complementary — research is interactive and bursty during working hours, training is batch and absorbs nights and weekends — and research runs at a preemptible priority below training so that idle training capacity is usable without becoming unreclaimable.

> **CONSEQUENCE**
>
> Research and Training run user-supplied images (Section 1.2) on shared nodes, co-tenant with other projects' code. Containment is the namespace template's responsibility: Pod Security admission level, prohibition of privileged containers and `hostPath` mounts, and a seccomp profile. See Section 14.

3.5 Project Provisioning

A project is desired state in PostgreSQL. A worker reconciles it into the cluster objects that constitute the project, from a template.

> **R-10**
>
> Project provisioning **MUST** be performed by a worker reconciling a project row, not by the API. The control plane **MUST NOT** call the Kubernetes API.

> **R-11**
>
> The provisioner **MUST** create, for each project and each cluster: the three workload namespaces; RoleBindings binding the appropriate controller identities; a ResourceQuota; a NetworkPolicy; a `LocalQueue` in each namespace; and the project's model-storage access. It **MUST** be idempotent, and **MUST** use Server-Side Apply with an explicit field manager.

Project creation is therefore eventually consistent: the API returns before the namespaces exist, and the operation's readiness is observable through the project's status.

4 Domain Model

4.1 Operations and Attempts

A logical operation may have several physical attempts:

```
Training operation
├── Attempt 1 → failed because node disappeared
├── Attempt 2 → failed because image was invalid
└── Attempt 3 → succeeded
```

R-12

Each attempt **MUST** receive the deterministic identity `echo-<operation-id>-a<attempt-number>`.

R-13

Retry creation **MUST** occur in a single PostgreSQL transaction that increments the attempt number. A worker **MUST NOT** independently invent a new attempt. This applies equally to an operator-initiated retry (Section 10.6).

Determinism is what makes crash recovery safe: a worker that dies after creating Kubernetes objects leaves objects that the next worker can find by name and adopt, rather than duplicating.

4.2 Custom Resources

Two custom resources model an operation in a cluster. The parent is operation-scoped and survives attempts; the child is attempt-scoped and is replaced with each one.

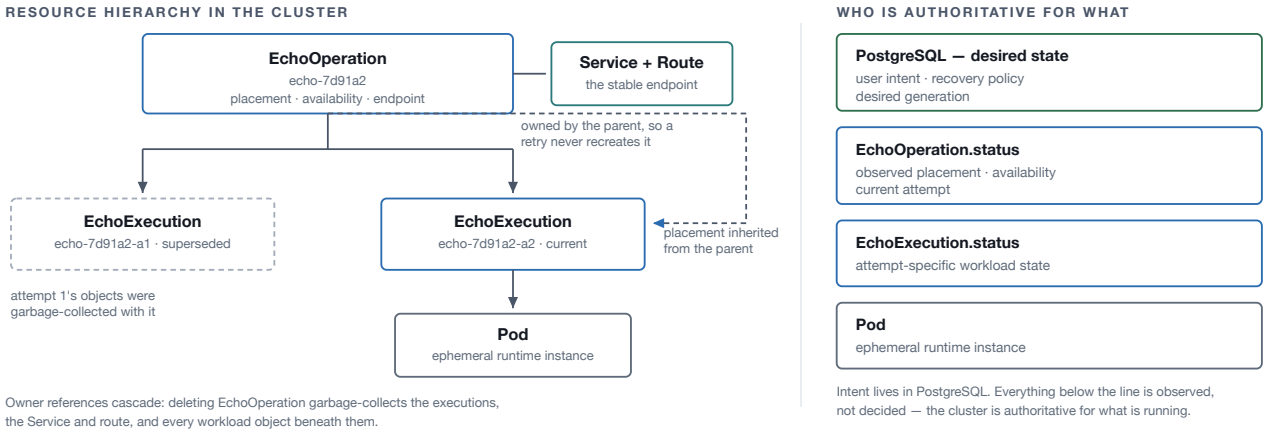

Figure 3. Resource hierarchy and the authority split. Anything that must outlive an attempt — placement, availability, the serving endpoint — belongs to `EchoOperation`. Anything specific to one execution belongs to `EchoExecution`.

4.2.1 EchoOperation

Operation-scoped. It owns the observed placement, the operation's availability, and a pointer to the current attempt.

```yaml
status:
  phase: Active
  placement:
    generation: 1
    cluster: cluster-a
    nodeName: gpu-node-07
    nodeUID: ...
  currentExecution:
    name: echo-7d91a2-a2
  conditions:
    - type: Ready
      status: "False"
      reason: PinnedNodeUnavailable
    - type: Degraded
      status: "True"
      reason: PinnedNodeUnavailable
```

4.2.2 EchoExecution

Attempt-scoped. It anchors one execution and owns the workload objects created for it.

```yaml
apiVersion: operations.echo.erez.io/v1alpha1
kind: EchoExecution
metadata:
  name: echo-7d91a2-a1
  namespace: echo-vision-training
  labels:
    echo.erez.io/operation-id: 7d91a2
    echo.erez.io/attempt: "1"
spec:
  operationID: 7d91a2
  operationGeneration: 4
  type: Training
  desiredState: Running
  queue: vision-compute
  priorityClass: standard-training
  resources:
    gpu:
      flavor: a100-80gb
      count: 8
  workload:
    image: registry.example.com/training/model:v12
    command: [...]
```

> **THE WORKLOAD STANZA IS TYPE-DEPENDENT**
>
> Above, `image` is the *submitter's* image — the Research and Training model. For Inference, `image` is E.C.H.O's own versioned runtime and `command` is not user-supplied at all; a declarative model specification takes its place. See Section 11.3.

R-14

Both custom resources are derived snapshots. Neither **MAY** be treated as the system of record for intent. PostgreSQL is authoritative for what was requested; the cluster is authoritative for what is running.

R-15

A new `EchoExecution` **MUST** inherit its placement from its parent `EchoOperation`, so that a pinned node survives attempt replacement.

R-16

Objects whose identity must outlive an attempt — in particular the inference Service and its external route — **MUST** be owned by `EchoOperation`. Objects specific to one execution **MUST** be owned by `EchoExecution`.

Owner references cascade: `EchoOperation` owns `EchoExecution`, which owns the Jobs, Services and PVCs of its attempt. Deleting the operation garbage-collects the whole tree.

5 Data Model and Persistence

5.1 Authority and Projection

PostgreSQL holds the original submission, the immutable execution plan, the queue, audit history, usage accounting, and projections of Kubernetes state. Once an `EchoExecution` exists, Kubernetes is authoritative for runtime state; PostgreSQL's `Dispatching`, `Running` and terminal records are projections of it.

> **R-17**
>
> An implementation **MUST NOT** treat a PostgreSQL runtime record as independent truth about what is running. Reconciliation decisions **MUST** be driven by observed cluster state.

5.2 Core Tables

TABLE	ROLE
`compute_operation`	The submitted operation and its policy snapshot
`operation_revision`	Versions of desired state
`reconcile_queue`	The durable work queue workers claim from
`operation_event`	Audit and event trail, including milestones and approvals

5.3 The Four Counters

Four independent counters exist. Conflating any two of them breaks a different guarantee, and they are the most common source of confusion in implementations.

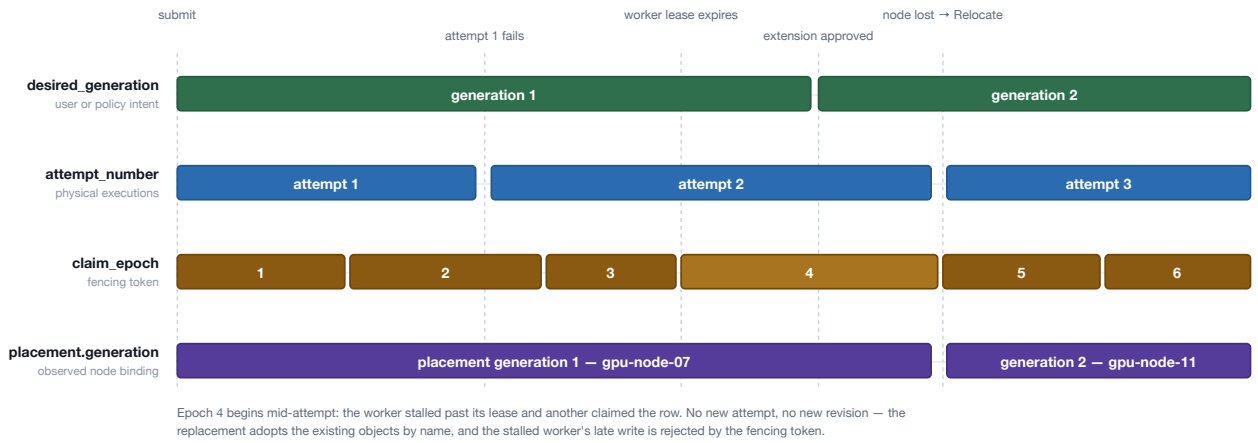

Figure 4. The four counters over one operation's life. Note the nesting: several attempts run under a single placement generation, and several claim epochs may occur within one attempt. A worker stalling and being replaced moves only `claim_epoch`.

COUNTER	INCREMENTS WHEN	LIVES IN
`desired_generation`	The user or policy changes intent	`compute_operation`, `reconcile_queue`
`claim_epoch`	A worker claims or re-claims the queue row	`reconcile_queue`
`attempt_number`	A physical execution failed and is retried	Attempt identity and CR name
`placement.generation`	A `Relocate` binding moves to a new node	`EchoOperation.status`

> **R-18**
>
> A worker that stalls and is replaced **MUST** cause only `claim_epoch` to advance. It **MUST NOT** produce a new revision, because desired state has not changed, and **MUST NOT** produce a new attempt: the replacement worker **MUST** adopt the existing objects by their deterministic names.

The first three counters live in PostgreSQL, which owns desired state and the queue. The fourth lives in `EchoOperation.status`, because an observed node binding is runtime state that the cluster owns. Revision 1 exists before any worker sees the operation: the submission transaction creates it.

5.4 Lifecycle Enforcement

The operation state machine is enforced in the database, not in application code. This is not an optimization: the control plane and the worker are written in different languages, and an invariant implemented twice is an invariant that will eventually disagree with itself.

> **R-19**
>
> A transition function in PostgreSQL **MUST** be the write path for operation state changes. It **MUST** provide a create path as well as a transition path, because submission is itself an entry into the state machine. It **MUST** accept an expected version and reject the write when the version does not match.

> **R-20**
>
> A trigger **MUST** enforce the terminal-state rule independently of the transition function, so that an ad-hoc `UPDATE` cannot move an operation backwards out of `Succeeded`, `Failed`, `Cancelled`, `Expired` or `Rejected`.

5.5 Schema Management

Because the schema carries real logic — the transition function, the trigger, partial indexes on the queue hot path, and the semantics of the fencing columns — it is authored deliberately rather than generated from object-relational models.

R-21

Migrations **MUST** be hand-written. Object-relational models **MUST NOT** be the source of the schema. A drift check between models and the live schema **SHOULD** run in continuous integration.

R-22

The control plane **MUST** own migration execution. Workers **MUST NOT** run migrations, and **MUST** verify a schema version at startup and refuse to start if it is older than they require.

6 Submission

6.1 Authentication

Authentication is performed against LDAP. LDAP offers bind and search but no portable signed assertion, so there is no token for the API to validate per request. The API therefore binds against the directory at login and issues its own session.

R-23

The API **MUST** issue an opaque, server-side session recorded in PostgreSQL. The session **MUST** be revocable with immediate effect. Self-contained bearer tokens that cannot be revoked **MUST NOT** be used.

R-24

`owner_subject` **MUST** hold an immutable directory identifier — `objectGUID` on Active Directory, `entryUUID` on OpenLDAP. It **MUST NOT** hold a distinguished name, which changes when a user moves organizational unit, nor `sAMAccountName` or `uid`, which can be renamed and reused. Distinguished name and username **MAY** be stored for display only.

> **WHY THIS ONE MATTERS**
>
> `owner_subject` is half of the idempotency key. A mutable subject means that renaming a user silently breaks idempotency for that user — a retried submission would create a duplicate operation rather than returning the original.

R-25

All directory connections **MUST** use LDAPS or StartTLS; simple bind transmits credentials in cleartext. Bind attempts **MUST** be rate-limited, because a retry loop against a directory with an account-lockout policy will lock out real users.

R-26

Group resolution **MUST** account for nested groups. On Active Directory this requires `LDAP_MATCHING_RULE_IN_CHAIN` (OID `1.2.840.113556.1.4.1941`) or a recursive search; a flat `memberOf` read misses indirect membership.

6.1.1 Directory synchronization

A synchronizer mirrors users and group membership into PostgreSQL on an interval, and a user's groups are re-resolved at login. Authorization is therefore a local join, and a directory outage does not stop submissions.

R-27

The synchronizer **MUST** revoke the sessions of users who have been disabled or removed in the directory. With no external identity provider, this is the only mechanism that enforces deprovisioning.

6.2 Idempotency

R-28

Submission **MUST** require a client idempotency key, and the schema **MUST** enforce `UNIQUE (owner_subject, idempotency_key)`. A retried request **MUST** return the original operation rather than creating a duplicate.

The conflict is resolved without aborting the transaction:

```
INSERT INTO compute_operation (...)
ON CONFLICT (owner_subject, idempotency_key) DO NOTHING
RETURNING operation_id;
-- no row returned: select and return the existing operation
```

6.3 The Submission Transaction

R-29

Submission **MUST** be a single PostgreSQL transaction. If it fails, nothing **MUST** have been submitted.

```
BEGIN;

INSERT INTO compute_operation (...);
INSERT INTO operation_revision (...);
INSERT INTO reconcile_queue (...);
INSERT INTO operation_event (...);

COMMIT;
```

The API responds `202 Accepted` with the operation identifier. Planning, admission and execution proceed asynchronously.

6.4 Policy and Approval

Policy is data in PostgreSQL — per-group quotas, maximum durations, priority ceilings and permitted GPU flavors — maintained by platform administrators through the E.C.H.O administrative interface.

R-30

The resolved policy decision — groups, limits, priority ceiling and approved duration — **MUST** be snapshotted onto the operation at submission.

R-31

Platform limits **MUST** be evaluated by the policy engine before a workload reaches Kueue. Kueue enforces cluster quota only and **MUST NOT** be relied upon to apply organizational policy.

6.4.1 Approved duration and exceptions

`approved_duration` defaults to the policy limit for the requesting group and workload type. Requests within the limit require no human involvement. An approver may grant more than the limit.

R-32

An approval that exceeds a policy limit **MUST** be recorded as an explicit exception in `operation_event`, carrying the approver's identity and the limit exceeded. It **MUST NOT** be folded silently into `approved_duration` as though policy had permitted it.

R-33

An extension request **MUST** be evaluated against policy and group membership as they stand at the time of the request, not against the snapshot taken at submission. The resulting decision **MUST** be recorded as an `operation_event`.

A tightened limit therefore takes effect immediately, and a user who has left a team cannot continue extending on that team's allowance. Because an extension can be refused that would have been granted earlier, the refusal message **SHOULD** state which limit applied.

7 Planning

7.1 Cluster Selection

The submitter names the target cluster. The planner validates that choice against policy and flavor availability and writes it into the plan; it does not rank candidates and does not balance load.

> **R-34**
> The planner **MUST** validate that the requested cluster is registered, that the requesting group is permitted to use it, and that it offers the requested GPU flavor. It **MUST NOT** substitute a different cluster.

> **R-35**
> The planner **MUST NOT** select a node and **MUST NOT** compute free GPU capacity. Node selection belongs to the Kubernetes scheduler and quota admission belongs to Kueue.

Because the cluster is part of submitted intent, a retry that is re-planned validates the same cluster, so an operation cannot migrate to a cluster that lacks its data. The cost is that there is no automatic failover: if the chosen cluster is saturated or unhealthy, the operation waits.

> **R-36**
> The user interface **MUST** present per-cluster capacity, drawn from PostgreSQL projections, so that the submitter's choice is informed.

7.2 The Execution Plan

The plan is written once and never modified. It records target cluster, namespace, queue, priority, GPU flavor and attempt identity.

> **R-37**
> The execution plan **MUST** be immutable. A retry **MUST** receive its own plan, produced by re-planning, rather than mutating the previous one.

8 Worker Protocol

The worker protocol is the mechanical core of the system. It is expressed in SQL because the guarantees it provides — mutual exclusion, lease expiry and fencing — are database guarantees.

8.1 Claiming

> **R-38**
>
> Workers **MUST** claim on the PostgreSQL writer using row locking with `SKIP LOCKED`, so that competing consumers never select the same row.

```sql
WITH candidate AS (
    SELECT operation_id
    FROM reconcile_queue
    WHERE state = 'pending'
      AND available_at <= clock_timestamp()
    ORDER BY priority DESC, available_at
    FOR UPDATE SKIP LOCKED
    LIMIT 1
)
UPDATE reconcile_queue AS queue
SET state = 'claimed',
    lease_owner = $1,
    lease_until = clock_timestamp() + interval '30 seconds',
    claim_epoch = claim_epoch + 1
FROM candidate
WHERE queue.operation_id = candidate.operation_id
RETURNING queue.*;
```

The same statement is used by the planner and the expiry scanner. Nothing in it is specific to workers, which is why none of the three roles needs leader election.

8.2 Leases and Fencing

A lease bounds how long a claim is honoured. It does not stop a stalled worker from waking up and attempting to write — that is what the fencing token is for.

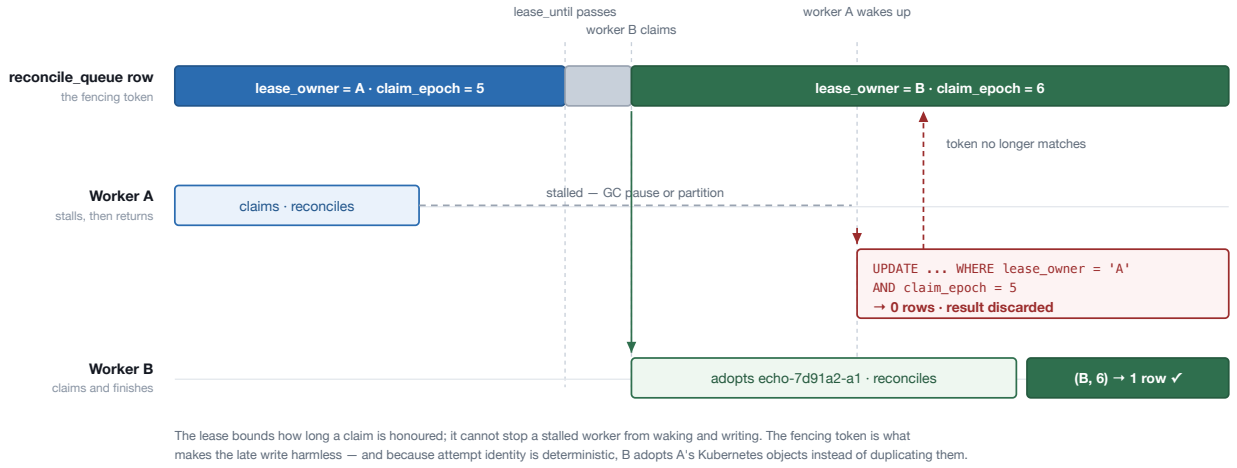

Figure 5. Lease expiry and fence rejection. Worker A stalls past its lease; worker B claims the row, incrementing `claim_epoch`. When A recovers and writes, its `WHERE` clause matches zero rows, and it discards its result rather than corrupting state.

R-39

Every completion update **MUST** carry the fencing token — `lease_owner` and `claim_epoch` — in its `WHERE` clause.

```
UPDATE reconcile_queue
SET processed_generation = $processed_generation,
    state = CASE
        WHEN desired_generation > $processed_generation
            THEN 'pending'
        ELSE 'idle'
    END,
    lease_owner = NULL,
    lease_until = NULL
WHERE operation_id = $operation_id
  AND lease_owner = $worker_id
  AND claim_epoch = $claim_epoch;
```

R-40

If the completion update affects zero rows, the worker **MUST** conclude that it has lost its lease and **MUST** discard its result. It **MUST NOT** retry the write, and **MUST NOT** take any further action on that operation under the old claim.

The `CASE` expression re-arms the row in the same statement that releases it: if desired state advanced while the worker was busy, the row returns to `pending` rather than `idle`, and the next claim picks up the newer intent.

8.3 Wake-up and Polling

R-41

`LISTEN` / `NOTIFY` **MAY** be used to wake workers promptly. It **MUST NOT** be treated as the durable queue: polling **MUST** remain the reliable path, and correctness **MUST NOT** depend on a notification being delivered.

8.4 Reconciliation

R-42

Kubernetes writes **MUST** use Server-Side Apply with an explicit field manager, so that ownership of each field is recorded and manual modification of an owned field is detected or overwritten.

R-43

Reconciliation **MUST** be level-triggered. The next action **MUST** be derived from observed cluster state, never from a record of which steps were previously completed. A reconciler that skips work because a milestone says it is done is incorrect the moment an object is deleted behind it.

R-44

A worker that finds an object matching the deterministic attempt identity **MUST** adopt it rather than creating a duplicate.

R-45

Periodic full reconciliation **MUST** run, so that a missed watch event is eventually corrected.

8.5 Orchestration Milestones

Milestones record the operation's orchestration progress — validation, planning, workload creation, admission and startup — so that an operator can see where an operation stands and an auditor can reconstruct what happened.

R-46

Milestones **MUST** be recorded in PostgreSQL and **MAY** be projected into `EchoOperation.status` for inspection. They **MUST NOT** determine control flow; see R-43.

Milestone recording is continuous, bound to the operation rather than to any Pod. Kubernetes may replace Pods without the operation restarting. Milestones do not restore application memory or computational progress inside a container, and are not intended to.

9 Scheduling and Admission

9.1 Three Layers

Placement is decided by three independent layers. Each has a responsibility and, just as importantly, a prohibition. The prohibitions are what keep E.C.H.O from racing Kubernetes.

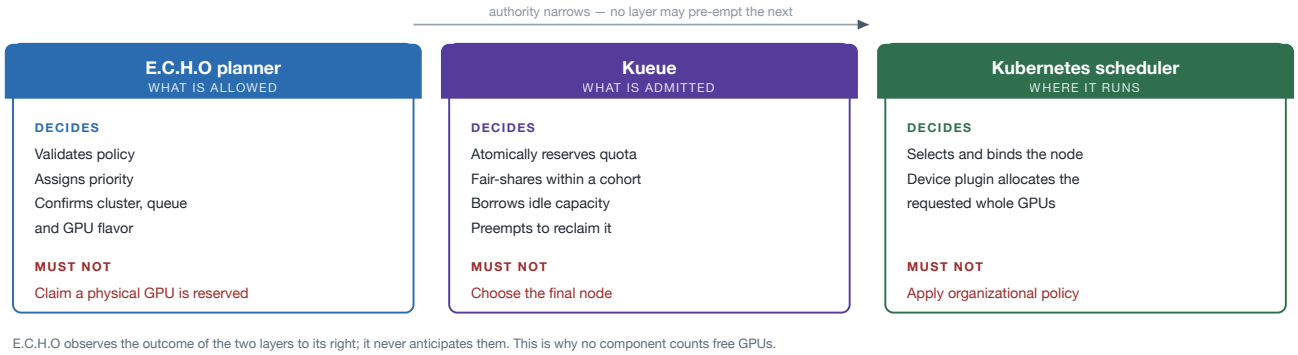

Figure 6. The three scheduling layers. Authority narrows from left to right: E.C.H.O decides what is allowed, Kueue decides what is admitted, and the Kubernetes scheduler decides where it runs. Nothing upstream may pre-empt a downstream decision.

> **R-47**
>
> E.C.H.O **MUST NOT** count currently free GPUs and **MUST NOT** represent a physical GPU as reserved. Any such count is a snapshot that is already stale, and acting on it races the scheduler.

9.2 Kueue Configuration

Each team has two ClusterQueues, in two cohorts, matching the node split from Section 3.4.

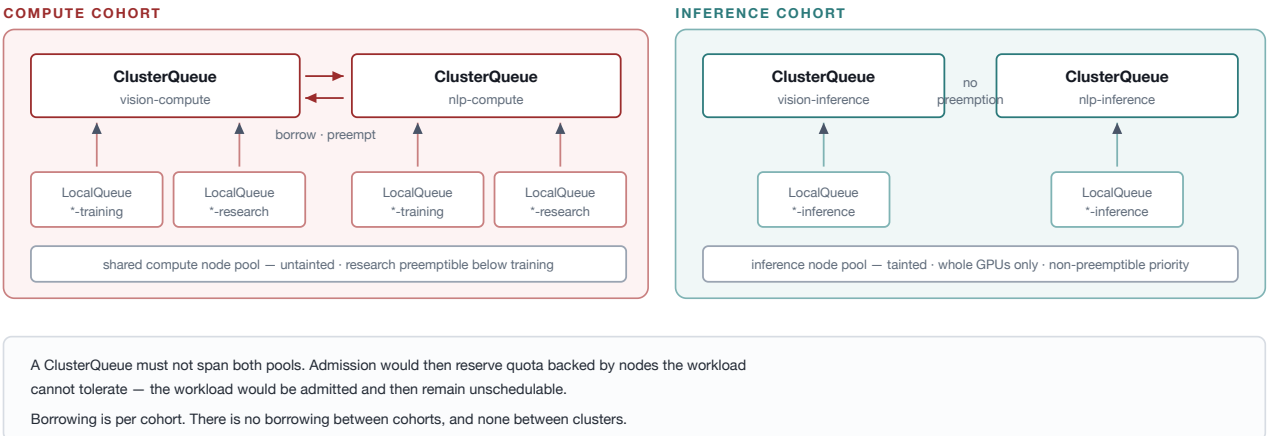

Figure 7. Queue and cohort layout. The compute cohort spans the shared training and research pool, so teams borrow each other's idle capacity and reclaim it by preemption. The inference cohort sits on the tainted pool and is never preempted.

R-48

Each project namespace **MUST** contain a `LocalQueue` pointing at the ClusterQueue for its team and workload class.

R-49

A ClusterQueue **MUST NOT** span node pools that a workload cannot use interchangeably. Admission would otherwise reflect capacity that is unreachable for the admitted workload.

R-50

Research workloads **MUST** run at a priority below training within the compute cohort, so that borrowed capacity is reclaimable.

9.3 Placement Observation

The worker applies the workload and then watches. It learns the node from the cluster; it never supplies one.

R-51

The worker **MUST** observe the node chosen by the scheduler and record it. It **MUST NOT** set `nodeName` to direct initial placement.

R-52

If placement does not succeed within the configured placement deadline, the operation **MUST** fail with reason `PlacementFailed`. The deadline is a policy parameter and **MUST** exceed realistic queue wait under Kueue.

10 Operation Lifecycle

10.1 States

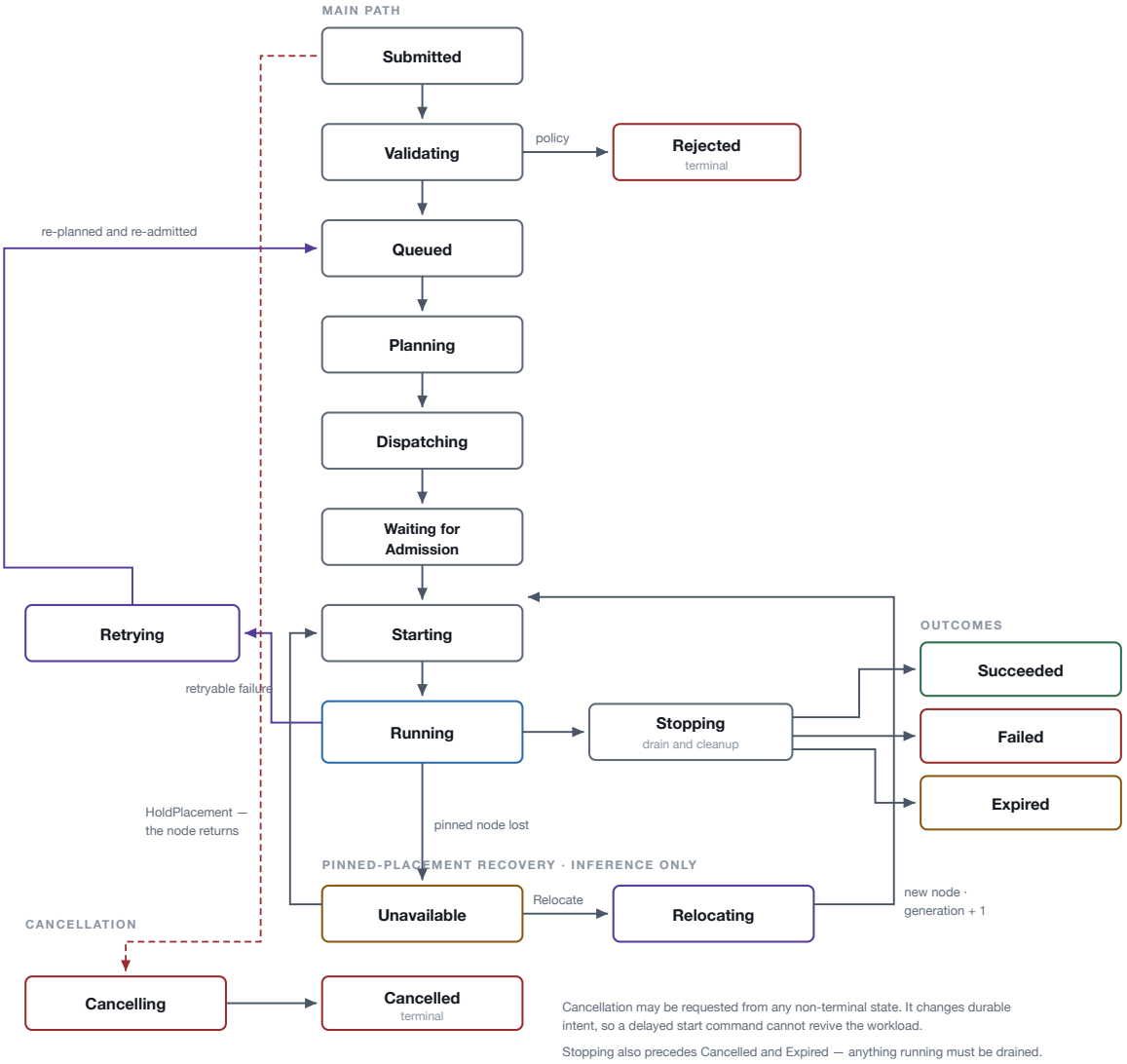

Figure 8. The operation lifecycle. Retry re-enters the queue rather than restarting execution, so every attempt is planned and admitted afresh. The `Unavailable` and `Relocating` states exist so that the loss of a pinned inference node is a recoverable condition rather than a terminal failure.

> **R-53**
>
> The terminal states are `Succeeded`, `Failed`, `Cancelled`, `Expired` and `Rejected`. An operation in a terminal state **MUST NOT** transition to any other state.

> **R-54**
>
> A retry **MUST** re-enter the lifecycle at `Queued`, not at `Starting`, so that it is re-planned and re-admitted.

10.2 Cancellation

Cancellation changes durable intent. It is not a message that races the workload.

> **R-55**
>
> Cancellation **MUST** be recorded as a change of desired state. A delayed start command **MUST NOT** revive a cancelled workload: commands are wake-up hints, and the worker **MUST** always read the newest desired state before acting.

10.3 Expiration and Extension

> **R-56**
>
> For time-bound operations, `expires_at` **MUST** be computed as `admitted_at + approved_duration`. The clock **MUST** start at admission, not at submission — time spent queueing is not time the user received.

> **R-57**
>
> Expiration and extension **MUST** use an expected database version, so that a stale expiry scanner cannot overwrite a newly approved extension.

10.4 Retry and Failure Classification

Whether a failure is retried depends on whether retrying could possibly help. Infrastructure failures are transient by nature; specification and application errors are not.

> **R-58**
>
> The following **MUST** be retried automatically, up to a policy-configured `max_attempts`:
>
> - Node loss
> - Infrastructure eviction or preemption
> - Spot interruption
> - Transient registry or network failure
> - Cluster capacity disappearing after placement

R-59

The following **MUST NOT** be retried automatically:

- Invalid image, or image not found
- Image authorization failure
- Invalid command or specification
- Application non-zero exit
- RBAC or policy rejection
- Deterministic out-of-memory from insufficient requested memory

R-60

A failure whose cause cannot be determined **MUST NOT** be retried automatically. The reason **MUST** be preserved, and an explicit user or operator retry **MUST** be permitted. Such a retry **MUST** still allocate its attempt number through the transaction required by R-13.

CLASSIFYING OUT-OF-MEMORY

Training and research share nodes, so an `OOMKilled` Pod may have been starved by a co-tenant rather than by its own request being too small. Classification **SHOULD** compare observed usage against the request and limit rather than relying on the kill reason alone.

10.5 Node Loss and Recovery Policy

An inference model has exactly one active placement. When its node is lost, the recovery policy decides whether the placement is held or moved. Either way, `Failed` is not used: a terminal state would make a maintenance reboot permanently destroy a production service.

```
HoldPlacement   Running → Unavailable → Starting → Running
                              │
                              └── wait for the pinned node to return

Relocate        Running → Unavailable → Relocating → Starting → Running
                              │
                              └── placement.generation += 1, new node permitted
```

R-61

Loss of a pinned node **MUST** place the operation in the non-terminal `Unavailable` state with reason `PinnedNodeUnavailable`. It **MUST NOT** be recorded as `Failed`.

> **R-62**
>
> Under `HoldPlacement` the binding **MUST** remain fixed for the operation's lifetime unless the user changes it. Under `Relocate` the binding **MUST** remain fixed within a placement generation, and relocation **MUST** increment `placement.generation`.

> **R-63**
>
> A `PinnedNodeUnavailable` condition that persists **MUST** raise an alert. Under `HoldPlacement` an operation will otherwise wait indefinitely on a node that may never return, presenting as managed while serving nothing.

Node loss for training and research does not use these states. Those workloads have no pinned placement to wait for, and follow the retry path of Section 10.4 instead.

11 Workload Types

The three workload types differ in how they are materialized, how long they live, and — most consequentially — in who supplies the container image.

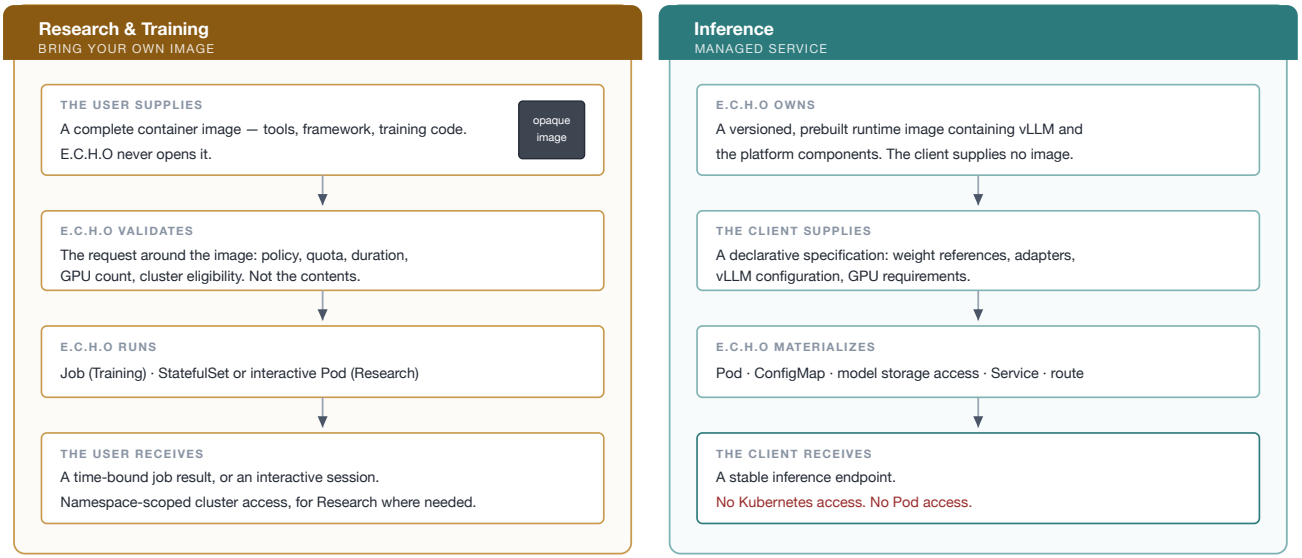

Figure 9. The two workload models. For Research and Training the user's image is opaque and E.C.H.O validates only the request around it. For Inference the runtime is E.C.H.O's, the client supplies a declarative specification, and what comes back is an endpoint rather than a Pod.

TYPE	KUBERNETES MATERIALIZATION	ADMISSION AND LIFETIME
Training	Job (JobSet when multi-node)	Kueue; time-bound; retry policy applies
Research	StatefulSet or interactive Pod, Service, PVC	Time-bound lease; preemptible priority
Inference	Deployment, Service, route, ConfigMap	No fixed expiry; bounded replicas and GPU quota

11.1 Training

Training is the first vertical slice of the platform, because it has an unambiguous beginning and a terminal result.

> **R-64**
>
> A training operation **MUST** be materialized as a Kubernetes Job admitted through Kueue. Multi-node training **SHOULD** use JobSet.

11.2 Research

A research environment is a time-bound interactive session. Expiration stops compute but does not immediately destroy the researcher's work.

```
Compute expires → Pod removed → PVC retained → PVC deleted after the retention period
```

R-65

Expiration of a research operation **MUST** release compute while retaining the PVC for the configured retention period.

R-66

Research is the only workload type for which namespace-scoped Kubernetes access **MAY** be granted to a project user, and only where interactive work genuinely requires it. Such access **MUST** be scoped to the project's research namespace and **MUST NOT** be cluster-wide.

11.3 Inference

Inference is a managed service. E.C.H.O owns and maintains a versioned, prebuilt runtime image containing vLLM and the required platform components.

R-67

The client **MUST** supply a declarative inference specification containing model weight references, any adapters or additional layers, vLLM configuration, and GPU requirements. The client **MUST NOT** supply a container image.

R-68

E.C.H.O **MUST** validate the specification against the option surface of the runtime version it will deploy, and **MUST** reject a specification it cannot validate. The validator is therefore version-coupled to the runtime image.

R-69

E.C.H.O **MUST** materialize the complete stack: the model Pod, its configuration, model storage access, the Service and the external route.

R-70

The client **MUST** receive a stable inference endpoint. End users **MUST NOT** receive Kubernetes or Pod access for managed inference. Platform administrators **MUST** retain direct access through Kubernetes RBAC for logging, diagnostics and maintenance.

Model weights are referenced rather than baked into the image, so cold start is a function of weight size and storage throughput. This is the same cost that a `Relocate` recovery incurs when a model must be allocated and loaded again.

11.3.1 Placement

The scheduling unit for inference is the model, not the Pod.

R-71

A model **MUST** be assigned to a single node and **MAY** use one or more whole GPUs on that node. GPU partitioning — MIG or time-slicing — **MUST NOT** be used.

R-72

Once placed, the assigned node **MUST** be recorded in `EchoOperation.status.placement` and **MUST** be fixed for the operation's lifetime, subject to the recovery policy of Section 10.5. Pods recreated by Kubernetes **MUST** return to that node.

R-73

The pin **MUST** be expressed as a node selector or affinity so that the scheduler continues to evaluate resources and taints. `nodeName` **MUST NOT** be used, because it bypasses the scheduler and produces an unschedulable Pod that fails opaquely.

R-74

Models requiring more than one node are outside the scope of this specification and **MUST NOT** be admitted as managed inference operations.

WHAT MANAGED INFERENCE DOES NOT PROVIDE

E.C.H.O does not provide zero-downtime inference, redundant replicas or active-active model deployment. Each model has exactly one active placement, and the recovery policy governs only whether that placement is held or moved. Workloads requiring redundancy are managed outside E.C.H.O.

12 End-to-End Flow

The following sequence is the reference for how the components interact. It shows the successful path; failure handling is specified in Section 15.

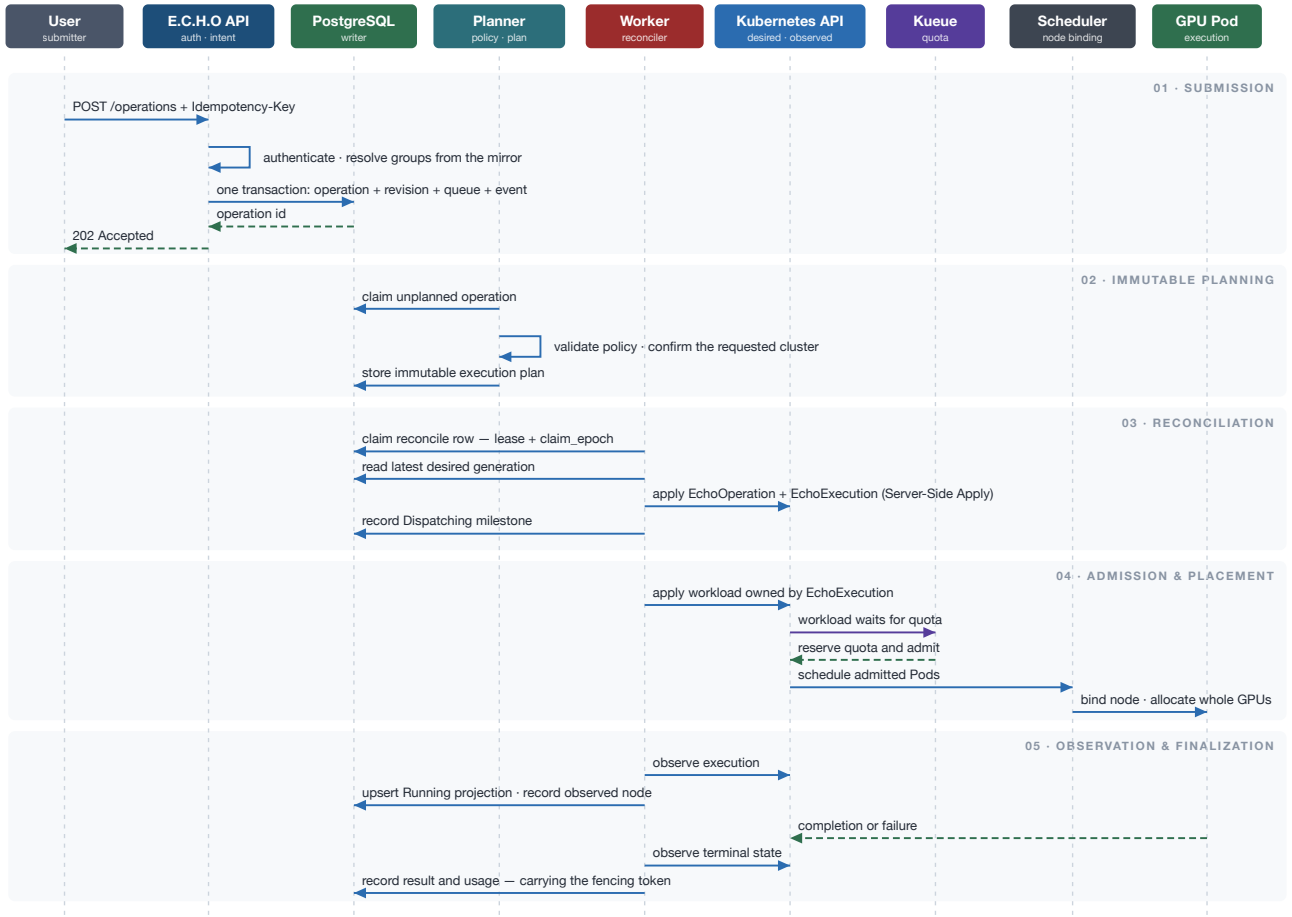

Figure 10. Submission through GPU execution. Note where authority changes hands: the planner commits a target, Kueue decides admission, the scheduler and device plugin decide the node, and from that point the worker observes rather than directs.

13 Observability and Accounting

13.1 Usage Accounting

Raw GPU telemetry is collected by Prometheus and DCGM. E.C.H.O periodically aggregates it into durable usage records in PostgreSQL; it does not duplicate the raw series.

R-75

Training and Research usage **MUST** be recorded per operation attempt and finalized when the attempt reaches a terminal state. Inference usage **MUST** be recorded continuously per model operation, because inference has no natural terminal state.

R-76

Allocated GPU time and observed utilization **MUST** be stored separately.

WHY THESE ARE NOT ONE NUMBER

Chargeback bills allocation; efficiency review reads utilization. Collapsing them hides the case the platform exists to prevent — an operation holding eight GPUs at four per cent for six hours is fully accounted for and almost entirely wasted. Only the pair makes that visible.

13.2 Retention

R-77

`operation_event`, `compute_operation` history and the usage tables **MUST** be partitioned by month and retained. Cold partitions **MAY** be detached and archived; they **MUST NOT** be discarded, because audit history is a first-class product of the platform.

R-78

Partition maintenance **MUST** be monitored. A partition job that stops silently is invisible until the hot set has grown by years.

14 Security Considerations

14.1 Credential Handling

Because authentication is LDAP-only, the API handles user credentials directly rather than delegating to an identity provider. This is a deliberate constraint of the deployment environment and it carries obligations.

> **R-79**
> Credentials **MUST NOT** be logged and **MUST NOT** be persisted in any form. All directory traffic **MUST** be encrypted, and bind attempts **MUST** be rate-limited (R-25).

14.2 Multi-Tenancy

Research and Training run user-supplied images. The platform's isolation posture rests on the namespace boundary, the node pool split, and the admission policy applied to those namespaces.

> **R-80**
> Workload namespaces **MUST** enforce a Pod Security admission level. Privileged containers and `hostPath` mounts **MUST NOT** be permitted, and a seccomp profile **MUST** be applied.

> **R-81**
> A NetworkPolicy **MUST** be applied to every workload namespace, so that a project's workloads cannot reach another project's Pods or Services.

14.3 Blast Radius

The three controller identities exist so that a compromise of one reconciler is contained. This property depends entirely on R-07 holding: if the identities are permitted to overlap "temporarily", the isolation is gone and nothing will report its absence.

> **R-82**
> Cluster credentials held by the worker pool **MUST** be scoped to the ServiceAccounts of Section 3.3 and **MUST NOT** be cluster-admin. Rotation **MUST** be possible without redeploying the control plane.

15 Failure Handling

Each row below is a failure the design anticipates and the mechanism that contains it. Where the mechanism is a requirement stated elsewhere, that requirement is the normative text.

FAILURE	RECOVERY
API retries submission	Idempotency key returns the original operation (R-28)
Two workers claim simultaneously	Row locking with `SKIP LOCKED` (R-38)
Worker pauses past its lease	Fencing token rejects the stale update (R-39, R-40)
Worker crashes after creating objects	Next worker finds the deterministic object and adopts it (R-44)
Transaction fails	Retry the complete transaction; nothing was partially applied (R-29)
PostgreSQL failover	Reconnect through the stable writer endpoint; reclaim expired leases
Kubernetes API unavailable	Back off and keep the operation pending
Watch event duplicated	Idempotent upsert with a unique event key
Watch event missed	Periodic full reconciliation (R-45)
Cancellation races with startup	The newest desired generation removes the workload (R-55)
Manual modification of a workload	Server-Side Apply field ownership detects or overwrites owned fields (R-42)
Read replica lags	Replicas are never used for correctness decisions (R-03)
Placement never succeeds	Placement deadline expires; `Failed` with `PlacementFailed` (R-52)
Pinned inference node lost	Non-terminal `Unavailable`; recovery policy decides (R-61, R-62)
Directory unavailable	Authorization reads the local mirror; submissions continue (Section 6.1.1)
User removed from the directory	Sync revokes their sessions at the next cycle (R-27)

> **THE PATTERN BEHIND THE TABLE**
>
> Almost every row resolves to one of three mechanisms: a database constraint, a fencing token, or re-deriving state from observation. None of them requires components to agree with each other, which is why they hold under partition — and why an implementation that replaces any of them with coordination will be correct in testing and wrong in production.

A Design Decisions

This appendix records the reasoning behind the normative text. Each entry states what was chosen, why, and what it costs. A decision recorded without a cost is a decision that was not examined.

A.1 Python control plane, Go worker pool

Decision. The API, planner, policy engine and scanner are Python; the worker pool is Go.

Rationale. The control plane never calls Kubernetes, so it gains nothing from a Kubernetes client library. The worker does, and Go is where the mature controller tooling lives: shared informer caching, resync, rate-limited requeue, typed Server-Side Apply, and first-class APIs for Kueue and the training operators. The split is cheap because PostgreSQL is already the contract between the two.

Trade-off. Two toolchains and two dependency streams; and the state machine would exist twice, which is why A.6 moves it into the database.

A.2 Web UI served by the API

Decision. React and TypeScript, built to static assets and served by the API process.

Rationale. One origin, no CORS, one deployable. The UI reaches PostgreSQL only through the API, which makes the read-replica prohibition structural rather than a rule to remember.

Trade-off. UI releases couple to API releases, and there is no server-side rendering.

A.3 The worker reconciles the custom resources

Decision. The worker applies the resources and reconciles them into workloads. There is no separate operator.

Rationale. One actor owns the chain, so there is one reconcile loop to debug and no second system to keep in step.

Trade-off. Nothing continues reconciling if the worker pool is down.

A.4 One control-plane package, three entrypoints

Decision. A single package exposing api, planner and scanner, deployed as separate Deployments.

Rationale. All three claim work with the same row-locking protocol, so all three scale horizontally and none needs leader election or a singleton guarantee.

Trade-off. The claiming query must be written as SQL rather than expressed through the object-relational layer.

A.5 Hand-written migrations

Decision. Migrations are authored by hand; object-relational models are a query layer, not the schema's author. A drift check runs in CI.

Rationale. The schema carries real logic — a transition function, a trigger, partial indexes on the queue hot path, fencing semantics. Generated migrations would either omit those objects or repeatedly propose to drop them.

Trade-off. No autogenerate convenience for routine column additions.

A.6 The lifecycle is enforced in PostgreSQL

Decision. A transition function is the write path; a trigger is the backstop against ad-hoc updates.

Rationale. The protocol is already expressed in SQL: queue transitions, the re-arm decision, fencing, and version-checked expiry are all statements. With two languages writing to one database, an invariant implemented in application code is an invariant implemented twice.

Trade-off. PL/pgSQL to maintain and review, and logic that is harder to unit test than a class.

A.7 LDAP only, with E.C.H.O-issued sessions

Decision. Authentication binds against the directory; the API issues its own revocable server-side session. owner_subject holds an immutable directory identifier.

Rationale. A deployment constraint. LDAP offers no portable signed assertion, so there is no token to validate per request and the API must own sessions. An immutable subject matters because it is half the idempotency key.

Trade-off. No single sign-on, and the API becomes a credential-handling surface.

A.8 Multi-cluster with cluster-agnostic workers

Decision. Any worker can materialize any plan in that plan's target cluster.

Rationale. Target cluster is part of the plan, so the fleet is addressable from one control plane without per-cluster control-plane deployments.

Trade-off. One process holds credentials for the whole fleet and a controller cache per cluster, so memory grows with cluster count and one unhealthy cluster must be prevented from starving the others.

A.9 Directory mirror with refresh at login

Decision. A synchronizer mirrors users and groups into PostgreSQL; groups are re-resolved at login.

Rationale. Authorization becomes a local join, so a directory outage does not stop submissions. Without a token carrying group claims, the alternative is querying the directory on the submission path.

Trade-off. A staleness window between cycles, which matters most for removal from a group.

A.10 One policy snapshot per operation

Decision. The resolved decision is snapshotted at submission. Extensions are re-evaluated against current policy and recorded as events.

Rationale. A tightened limit then takes effect immediately, and a user who has left a team cannot keep extending on its allowance.

Trade-off. An extension can be refused that would have been granted a day earlier, so the refusal must explain itself.

A.11 Policy binds; a human may exceed it

Decision. approved_duration defaults to the policy limit. An approver may grant more, recorded as an explicit exception.

Rationale. A hard ceiling with no override forces every genuine exception into a policy edit that then applies to the whole group. An override recorded as an exception keeps the limit meaningful and the exception visible.

Trade-off. An approval path and its interface, and a decision a human must now justify.

A.12 Per-project namespaces, two workload identities

Decision. echo-<project>-<type> namespaces; controller identities isolated in echo-access.

Rationale. The namespace is the unit that carries quota, storage, NetworkPolicy, RBAC and the queue. Splitting the identities moves the isolation boundary from the cluster to the authorization layer, so a fault in the compute reconciler cannot reach an inference service.

Trade-off. Many namespaces, and a RoleBinding set per project that must be provisioned.

A.13 A dedicated, tainted inference node pool

Decision. Inference nodes are tainted and inference runs non-preemptibly.

Rationale. Namespaces isolate authorization, not resources. Hard node separation is what actually protects the workload serving live traffic.

Trade-off. Idle inference nodes cannot absorb training work.

A.14 Project provisioning reconciles a row

Decision. A project is desired state in PostgreSQL; a worker reconciles it into namespaces, RBAC, quota, NetworkPolicy and queues, under a third cluster-scoped identity.

Rationale. Provisioning is a Kubernetes write, so doing it from the API would break the control-plane boundary. Reconciling from a row is idempotent on retry and gets field ownership for free. The separate identity means that whatever can create namespaces cannot run workloads.

Trade-off. Project creation is eventually consistent; the API returns before the namespaces exist.

A.15 Two ClusterQueues per team, in two cohorts

Decision. A compute queue over the shared pool and an inference queue over the tainted pool.

Rationale. A queue spanning both pools would admit against capacity the workload cannot use. Borrowing within the compute cohort is what turns idle training capacity into usable research capacity.

Trade-off. Preempted compute workloads must tolerate being killed.

A.16 Milestones are observability, not control flow

Decision. Orchestration progress is recorded continuously and bound to the operation, never to a Pod.

Rationale. Recording progress is useful; branching on it is not. A reconciler that skips work because a milestone says it is done is wrong the moment an object is deleted behind it. Re-deriving from observed state is correct by construction.

Trade-off. There is no lifecycle state that displays 'saving progress', and milestones do not make resumption faster — they make it auditable.

A.17 Failures are classified before they are retried

Decision. Infrastructure failures retry automatically; specification and application errors do not; unknown causes require an explicit retry.

Rationale. Retrying an invalid image cannot succeed; it only delays the user's error message and consumes GPU time.

Trade-off. A classifier to maintain, and out-of-memory is genuinely ambiguous on shared nodes.

A.18 The submitter chooses the cluster

Decision. The planner validates the requested cluster rather than ranking candidates.

Rationale. Predictable, and it keeps E.C.H.O away from anything resembling capacity arbitration. It also means a retry cannot migrate to a cluster lacking the operation's data.

Trade-off. No automatic failover, and hot and cold clusters, since nothing balances between them.

A.19 Per-cluster ServiceAccount tokens

Decision. Each cluster defines the controller identities; their tokens are held where the workers run.

Rationale. Simple and native, and RBAC stays enforced by each target cluster rather than by E.C.H.O.

Trade-off. One location holds fleet-wide credentials, and rotation is per-cluster.

A.20 The model is the inference scheduling unit

Decision. One model, one node, whole GPUs, pinned after placement. No partitioning, no multi-node models.

Rationale. Pinning is achieved by observing the scheduler's choice and remembering it, never by choosing a node, so no component counts free GPUs.

Trade-off. A single active placement means a model is unavailable during node loss or relocation.

A.21 Allocated and observed usage are stored apart

Decision. Aggregated into PostgreSQL on an interval; raw series stay in Prometheus.

Rationale. Chargeback bills allocation and efficiency review reads utilization. One number cannot serve both, and collapsing them hides idle-but-allocated GPUs.

Trade-off. Inference usage accrues indefinitely, so the usage tables need the same partitioning as the audit tables.

A.22 Audit history is retained

Decision. Monthly partitions, retained; cold partitions archived rather than dropped.

Rationale. Audit history is a product of the platform, not a byproduct.

Trade-off. Partition maintenance becomes a scheduled job that must not fail silently.

A.23 Sessions are opaque and revocable

Decision. The session is a row, killed instantly on logout or on directory removal.

Rationale. With no identity provider there is nothing else to enforce deprovisioning. A self-contained token would keep a dismissed user's access valid until expiry.

Trade-off. A database lookup per request, which is trivial next to the work being authorized.

A.24 Kubernetes is authoritative for runtime state

Decision. PostgreSQL owns intent, plan, queue, audit and usage; once an execution exists, the cluster owns what is running.

Rationale. This is the ownership table read from the other side. Stating it explicitly prevents the common error of treating a stale Running row as fact.

Trade-off. Reporting is a projection and can lag; correctness decisions must not read it.

A.25 Policy is data in PostgreSQL

Decision. Quotas, durations, priority ceilings and permitted flavors are rows, edited through the administrative interface.

Rationale. Keeping policy where the snapshot reads it means no external engine sits on the admission path, and policy changes are audited by the same machinery as everything else.

Trade-off. Genuinely conditional policy is awkward as rows, and there is no policy-as-code review flow.

A.26 An operation-scoped parent resource

Decision. EchoOperation parents the attempt-scoped EchoExecution and owns placement, availability and the endpoint.

Rationale. An attempt-scoped resource cannot own anything that must outlive an attempt. Placement is observed runtime state, so it belongs in the cluster rather than in PostgreSQL — the mirror image of where milestones belong.

Trade-off. A second custom resource and controller, and a parent/child status relationship that can disagree with itself.

A.27 Two workload models

Decision. Research and Training supply an opaque image; Inference is a managed service on E.C.H.O's runtime.

Rationale. The two want opposite things. Research and Training are open-ended and the user's code is the point. Inference is a product surface, and owning the runtime is what makes validation, upgrades and a durable endpoint possible at all.

Trade-off. A client wanting a different serving stack cannot have one, and E.C.H.O takes on permanent maintenance of a runtime image and a validator that must track it.

B Index of Requirements

Every numbered requirement in this specification, in order, with the section that states it in full. Entries are abbreviated; the section text is normative. An implementation is conformant when it satisfies all 82.

ID	§	REQUIREMENT
R-01	2.2	A worker MUST NOT hold a PostgreSQL transaction open while calling the Kubernetes API.
R-02	2.2	A worker process holding managers for multiple clusters MUST isolate them, so that one cluster's reconnect storm or cache failure cannot starve reconciliation for the others.
R-03	2.2	Read replicas MUST NOT be used for correctness decisions.
R-04	3.1	Each registered cluster MUST have an entry in the cluster registry recording its identifier, API endpoint, credential reference and available GPU flavors.
R-05	3.2	Workload namespaces MUST be named `echo-<project>-<type>` where *type* is one of `training`, `research` or `inference`.
R-06	3.2	Selection of E.C.H.O-managed objects MUST use labels, not namespace-name parsing.
R-07	3.3	The three controller identities MUST hold disjoint permissions. `echo-project-controller` MUST NOT hold verbs on Pods, Jobs or Deployments, and the two workload identities MUST NOT …
R-08	3.3	Roles and RoleBindings MUST be created in the target namespace with subjects referring to the ServiceAccounts in `echo-access`.
R-09	3.4	Inference nodes MUST be tainted, and only inference workloads MUST tolerate that taint.
R-10	3.5	Project provisioning MUST be performed by a worker reconciling a project row, not by the API. The control plane MUST NOT call the Kubernetes API.
R-11	3.5	The provisioner MUST create, for each project and each cluster: the three workload namespaces; RoleBindings binding the appropriate controller identities; a ResourceQuota; a NetworkPolicy; a …
R-12	4.1	Each attempt MUST receive the deterministic identity `echo-<operation-id>-a<attempt-number>`.
R-13	4.1	Retry creation MUST occur in a single PostgreSQL transaction that increments the attempt number.
R-14	4.2	Both custom resources are derived snapshots.
R-15	4.2	A new `EchoExecution` MUST inherit its placement from its parent `EchoOperation`, so that a pinned node survives attempt replacement.
R-16	4.2	Objects whose identity must outlive an attempt — in particular the inference Service and its external route — MUST be owned by `EchoOperation`. Objects specific to one …
R-17	5.1	An implementation MUST NOT treat a PostgreSQL runtime record as independent truth about what is running.
R-18	5.3	A worker that stalls and is replaced MUST cause only `claim_epoch` to advance.
R-19	5.4	A transition function in PostgreSQL MUST be the write path for operation state changes.
R-20	5.4	A trigger MUST enforce the terminal-state rule independently of the transition function, so that an ad-hoc `UPDATE` cannot move an operation backwards out of …
R-21	5.5	Migrations MUST be hand-written.
R-22	5.5	The control plane MUST own migration execution.
R-23	6.1	The API MUST issue an opaque, server-side session recorded in PostgreSQL. The session MUST be revocable with immediate effect.

R-24	6.1	`owner_subject` MUST hold an immutable directory identifier — `objectGUID` on Active Directory, `entryUUID` on OpenLDAP. It MUST NOT hold a …
R-25	6.1	All directory connections MUST use LDAPS or StartTLS; simple bind transmits credentials in cleartext.
R-26	6.1	Group resolution MUST account for nested groups.
R-27	6.1	The synchronizer MUST revoke the sessions of users who have been disabled or removed in the directory.
R-28	6.2	Submission MUST require a client idempotency key, and the schema MUST enforce `UNIQUE (owner_subject, idempotency_key)`. A retried request MUST return the original operation rather …
R-29	6.3	Submission MUST be a single PostgreSQL transaction.
R-30	6.4	The resolved policy decision — groups, limits, priority ceiling and approved duration — MUST be snapshotted onto the operation at submission.
R-31	6.4	Platform limits MUST be evaluated by the policy engine before a workload reaches Kueue.
R-32	6.4	An approval that exceeds a policy limit MUST be recorded as an explicit exception in `operation_event`, carrying the approver's identity and the limit exceeded.
R-33	6.4	An extension request MUST be evaluated against policy and group membership as they stand at the time of the request, not against the snapshot taken at submission.
R-34	7.1	The planner MUST validate that the requested cluster is registered, that the requesting group is permitted to use it, and that it offers the requested GPU flavor.
R-35	7.1	The planner MUST NOT select a node and MUST NOT compute free GPU capacity.
R-36	7.1	The user interface MUST present per-cluster capacity, drawn from PostgreSQL projections, so that the submitter's choice is informed.
R-37	7.2	The execution plan MUST be immutable.
R-38	8.1	Workers MUST claim on the PostgreSQL writer using row locking with `SKIP LOCKED`, so that competing consumers never select the same row.
R-39	8.2	Every completion update MUST carry the fencing token — `lease_owner` and `claim_epoch` — in its `WHERE` clause.
R-40	8.2	If the completion update affects zero rows, the worker MUST conclude that it has lost its lease and MUST discard its result.
R-41	8.3	`LISTEN`/`NOTIFY` MAY be used to wake workers promptly.
R-42	8.4	Kubernetes writes MUST use Server-Side Apply with an explicit field manager, so that ownership of each field is recorded and manual modification of an owned field is detected or overwritten.
R-43	8.4	Reconciliation MUST be level-triggered.
R-44	8.4	A worker that finds an object matching the deterministic attempt identity MUST adopt it rather than creating a duplicate.
R-45	8.4	Periodic full reconciliation MUST run, so that a missed watch event is eventually corrected.
R-46	8.5	Milestones MUST be recorded in PostgreSQL and MAY be projected into `EchoOperation.status` for inspection.

R-47	9.1	E.C.H.O MUST NOT count currently free GPUs and MUST NOT represent a physical GPU as reserved.
R-48	9.2	Each project namespace MUST contain a `LocalQueue` pointing at the ClusterQueue for its team and workload class.
R-49	9.2	A ClusterQueue MUST NOT span node pools that a workload cannot use interchangeably.
R-50	9.2	Research workloads MUST run at a priority below training within the compute cohort, so that borrowed capacity is reclaimable.
R-51	9.3	The worker MUST observe the node chosen by the scheduler and record it.
R-52	9.3	If placement does not succeed within the configured placement deadline, the operation MUST fail with reason `PlacementFailed`. The deadline is a policy parameter and MUST exceed …
R-53	10.1	The terminal states are `Succeeded`, `Failed`, `Cancelled`, `Expired` and `Rejected`. An operation in a terminal state MUST NOT …
R-54	10.1	A retry MUST re-enter the lifecycle at `Queued`, not at `Starting`, so that it is re-planned and re-admitted.
R-55	10.2	Cancellation MUST be recorded as a change of desired state.
R-56	10.3	For time-bound operations, `expires_at` MUST be computed as `admitted_at + approved_duration`. The clock MUST start at admission, not at submission — time spent …
R-57	10.3	Expiration and extension MUST use an expected database version, so that a stale expiry scanner cannot overwrite a newly approved extension.
R-58	10.4	The following MUST be retried automatically, up to a policy-configured `max_attempts`:.
R-59	10.4	The following MUST NOT be retried automatically:.
R-60	10.4	A failure whose cause cannot be determined MUST NOT be retried automatically.
R-61	10.5	Loss of a pinned node MUST place the operation in the non-terminal `Unavailable` state with reason `PinnedNodeUnavailable`. It MUST NOT be recorded as `Failed`.
R-62	10.5	Under `HoldPlacement` the binding MUST remain fixed for the operation's lifetime unless the user changes it.
R-63	10.5	A `PinnedNodeUnavailable` condition that persists MUST raise an alert.
R-64	11.1	A training operation MUST be materialized as a Kubernetes Job admitted through Kueue.
R-65	11.2	Expiration of a research operation MUST release compute while retaining the PVC for the configured retention period.
R-66	11.2	Research is the only workload type for which namespace-scoped Kubernetes access MAY be granted to a project user, and only where interactive work genuinely requires it.
R-67	11.3	The client MUST supply a declarative inference specification containing model weight references, any adapters or additional layers, vLLM configuration, and GPU requirements.
R-68	11.3	E.C.H.O MUST validate the specification against the option surface of the runtime version it will deploy, and MUST reject a specification it cannot validate.
R-69	11.3	E.C.H.O MUST materialize the complete stack: the model Pod, its configuration, model storage access, the Service and the external route.

R-70	11.3	The client MUST receive a stable inference endpoint.
R-71	11.3	A model MUST be assigned to a single node and MAY use one or more whole GPUs on that node.
R-72	11.3	Once placed, the assigned node MUST be recorded in `EchoOperation.status.placement` and MUST be fixed for the operation's lifetime, subject to the recovery policy of Section 10.5.
R-73	11.3	The pin MUST be expressed as a node selector or affinity so that the scheduler continues to evaluate resources and taints. `nodeName` MUST NOT be used, because it bypasses the …
R-74	11.3	Models requiring more than one node are outside the scope of this specification and MUST NOT be admitted as managed inference operations.
R-75	13.1	Training and Research usage MUST be recorded per operation attempt and finalized when the attempt reaches a terminal state.
R-76	13.1	Allocated GPU time and observed utilization MUST be stored separately.
R-77	13.2	`operation_event`, `compute_operation` history and the usage tables MUST be partitioned by month and retained.
R-78	13.2	Partition maintenance MUST be monitored.
R-79	14.1	Credentials MUST NOT be logged and MUST NOT be persisted in any form.
R-80	14.2	Workload namespaces MUST enforce a Pod Security admission level.
R-81	14.2	A NetworkPolicy MUST be applied to every workload namespace, so that a project's workloads cannot reach another project's Pods or Services.
R-82	14.3	Cluster credentials held by the worker pool MUST be scoped to the ServiceAccounts of Section 3.3 and MUST NOT be cluster-admin.